Lab 8 – Wi-Fi on the ESP32-S3: Security analysis

Johan Torstensson
2026-11-11

1. Purpose and scope

This document analyses the security of Lab 8 (Wi-Fi on the ESP32-S3): which assets need protection, which threats exist, which measures have been taken, and which risks remain. It also describes how the protection of Wi-Fi credentials evolved during the lab, and what the next level would be.

Scope: the Lab 8 programs (`lab8_wifi_scan.cpp`, `lab8_wifi_connect.cpp`, planned web page and printer programs), the credential store `include/lab8_credentials.h`, the networks they use and the GitHub repository.

2. System overview

Component	Role	Connection
ESP32-S3 (`lab8-esp32`)	Scans, connects, checks internet, serves a web page, prints	Wi-Fi 2.4 GHz
ZTE MC888 Pro router	2.4 GHz network with internet access	WPA2
Internet server (`gstatic.com`)	Answers the internet check with HTTP 204	HTTP over the router
HP OfficeJet 8010	Prints the scan report	Own Wi-Fi Direct network (WPA2), port 9100
PC	Builds, uploads, enters credentials, runs smoke-tests	USB Serial
GitHub repository	Source code, plans, PR log	HTTPS

Data flows:

1. PC → ESP32 over USB Serial: firmware, credentials (once), test output.
2. ESP32 ↔ router: Wi-Fi traffic, DHCP, DNS.
3. ESP32 → internet: connectivity check.
4. ESP32 → printer: plain-text print job on port 9100.
5. PC → GitHub: source code and documentation.

3. Assets

Asset	Why it matters
Router Wi-Fi password	Gives access to the home/summerhouse network and every device on it
Printer Wi-Fi Direct password	Gives access to the printer
Network details (IPs, SSIDs)	Low value on their own; help an attacker map the network
Firmware integrity	Modified firmware could leak credentials or misuse the network
Repository contents	Public on GitHub; anything committed is readable forever

4. Threat actors

Actor	Capability	Realistic?
A. Repository reader	Reads everything ever committed to GitHub	Yes – most likely leak path
B. Outsider in radio range	Captures Wi-Fi traffic, does not know the Wi-Fi password	Yes
C. Device on the same network	Knows the Wi-Fi password; sees unencrypted traffic on the LAN	Possible (guests, other devices)
D. Physical access to the board	Can read flash over USB, open the setup menu	Possible for an unattended board
E. Attacker who flashes own firmware	Runs any code on the board	Unlikely for a lab board

5. Protection layers

Layer	Protects	Does not protect
WPA2/WPA3	All Wi-Fi traffic against actor B	Traffic against actor C (inside the network)
TLS (HTTPS)	Content of one connection end-to-end	Server name, DNS lookups, IPs, timing, stored data
`.gitignore` + review	Secrets leaking to GitHub (actor A)	Anything already committed
Credential store on the board	Secrets in source/firmware; casual flash reading	A determined actor D or E

No single layer covers everything; the measures below combine them.

6. Evolution of credential protection

Level	Where the password is	Protects against	Status
0	Plain text in the source code	Nothing	Avoided from the start
1	Plain text in `include/wifi_credentials.h`, ignored by git	Leaks via GitHub	8.1
2	XOR-obfuscated in ESP32 flash (NVS), typed in over Serial	+ secrets in source/firmware, readable flash dumps, copying to another board	**8.4b (current)**
3	AES-GCM in flash, key hidden in a read-protected eFuse	+ decoding with source code and MAC	Documented (section 10)
4	Flash + NVS encryption + Secure Boot	+ attacker's own firmware on the board	Documented (section 10)

7. Current implementation (level 2)

7.1 How it works

Credentials are typed in once over Serial and stored in the ESP32's NVS (non-volatile key-value storage in flash). No file on the PC holds them.

```
You type in the serial monitor
        |
        v
lab8_credentials.h --XOR with chip-specific keystream--> ESP32 flash (NVS)
                                                                  |
Every boot: read from flash --XOR back--> RAM --> WiFi.begin()
```

- First boot with empty NVS opens a setup menu automatically; later, pressing `c` within 3 s after reset opens it.
- Input is validated: SSID 1–32 characters, password 8–63, IPv4 address, port 1–65535.
- Typed passwords are echoed as `*`; the menu shows passwords only as "(set, hidden)". Serial output reports only `CRED_STATUS router=1 printer=1`.
- Credentials survive power-off, reset and new uploads; menu 4 or `pio run -t erase` deletes them.

7.2 The obfuscation

Each SSID and password is XORed with a keystream before it is written to flash, and XORed again when read:

1. **Seed** = FNV-1a hash of a fixed salt, the chip's 48-bit eFuse MAC address and the NVS key name (e.g. `router_pass`).
2. **Keystream** = successive values of a xorshift32 generator started from the seed; one byte per character.
3. **Stored value** = character XOR keystream byte.

Including the key name means two equal values (for example the same password for router and printer) are stored differently. Including the MAC means a flash image copied to another ESP32 does not decode.

7.3 What level 2 protects – and what it does not

Protects against	Does not protect against
Passwords in git, in the source or in the `.bin` firmware file	Actor D with the source code and the chip's MAC (readable with `esptool`)
Readable SSIDs/passwords in a flash dump (`strings`)	Actor E: own firmware calling `loadCredentials()`
A flash copy working on another ESP32	

Assessment: this is obfuscation, not encryption. By Kerckhoffs's principle a system must stay secure even when everything except the key is public. Here the algorithm is public (the repository), and the "key" is derived from the MAC address, which is not secret. Level 2 therefore raises the effort for a casual attacker but gives no protection against a determined one.

8. Analysis: why MAC-XOR fails for transmission

During the lab the idea came up to send data in the open as `data XOR MAC`. This does not work:

1. **The key is public.** The MAC address is sent in clear in the header of every Wi-Fi frame. A listener receives both the ciphertext and the key.
2. **Decoding is one operation.** Example with one byte (MAC byte `0xA4` is illustrative):

Step	Value
Plain character `T`	`0x54`
Sent: `0x54 XOR 0xA4`	`0xF0`
Listener: `0xF0 XOR 0xA4` (MAC from the same frame)	`0x54 = T`

3. **The key is short and repeats.** A MAC is 6 bytes. In longer data the key repeats every 6 bytes, so any guessed part of the content (e.g. that an SSID starts with `Tele2_`) reveals the key directly.
4. **The key never changes.** Equal messages give equal ciphertext, so a listener sees repetitions.

Conclusion: data in transit needs real encryption with a secret, per-connection key – in practice **TLS**. The ESP32 Arduino core provides it through `WiFiClientSecure` (mbedTLS with hardware AES/SHA). Certificates must be verified with `setCACert()`; `setInsecure()` encrypts but accepts any server, which leaves the connection open to a man-in-the-middle.

9. Analysis: the print job

The ESP32 sends the scan report to the printer as plain text on port 9100 (JetDirect), without encryption. This is a deliberate decision, not an oversight:

1. **The purpose is a readable paper copy.** The report exists to put the scan result on paper so that a person can read it. Whoever can see the page can read it; encrypting the path to the printer cannot protect data whose intended end state is readable text on paper.
2. **The content is not secret.** The report holds SSIDs, signal strength, channels and security modes – information that every Wi-Fi device in range already receives from the networks' broadcast beacons.
3. **The path is already limited.** The ESP32 joins the printer's own Wi-Fi Direct network, which is protected by WPA2, and only the printer and the ESP32 are on it. An outsider in radio range (actor B) cannot read the job.
4. **The printer only prints.** Port 9100 receives text to print; nothing sensitive is sent back.

What must still hold: **a print job must never contain credentials.** The report is built only from scan results; the Wi-Fi Direct password is used to join the printer's network and is never part of the printed text.

Conclusion: encrypted printing (IPP over TLS) would add complexity without protecting anything of value. The unencrypted print channel is accepted.

10. Next level (not implemented)

10.1 Level 3 – AES-GCM with a hidden eFuse key

1. Once, on the PC: burn a random 256-bit key into one of the ESP32-S3's six eFuse key blocks with `espefuse.py`, purpose `HMAC_UP`, read-protected. Neither software nor `esptool` can read it afterwards.
2. In the firmware: the chip's HMAC peripheral derives an AES-256 key from the hidden key. Credentials are stored with **AES-GCM**, which both encrypts and detects tampering, with a new random nonce for every write.
3. Only `lab8_credentials.h` changes; the menu and the programs stay the same.

Result: a flash dump is useless even with the source code and the MAC.

Cost and limits:

- The eFuse change is **permanent** and uses one key block.
- Actor E can still flash firmware that asks the HMAC peripheral to decrypt.

10.2 Level 4 – Flash encryption, NVS encryption and Secure Boot

Espressif's official route encrypts the whole flash and only accepts signed firmware, which also stops actor E. It burns several eFuses, fits poorly with PlatformIO's Arduino setup and can make the board unusable if done wrong.

10.3 Why level 2 was chosen

The boards are lab boards kept at home. An attacker with physical access is already inside the house and could usually reach the router directly. Levels 3 and 4 involve permanent hardware changes with a risk of damaging the board, for little real gain. For the unattended friggebod board, a **separate guest/IoT network** limits the damage of a stolen board more effectively: only that network's password is exposed, and it can be changed without touching other devices.

11. Risk register

Likelihood (L) and impact (I): High / Medium / Low.

ID	Risk	L	I	Measure	Decision / remaining risk
R1	Passwords committed to GitHub	M	H	No secrets in source; `.gitignore` covers `include/wifi_credentials*.h`; `git status` before commit	Fixed
R2	Passwords shown on Serial, LCD or web page	M	H	Never printed; input echoed as `*`; smoke-tests can assert absence	Fixed
R3	Passwords read from a stolen board	L	H	Level 2 obfuscation; guest network for unattended boards	Reduced; level 3 documented
R4	Internet check read or altered in transit	L	L	Planned: HTTPS with certificate verification	Open (issue I-1)
R5	DNS lookups, server name, IPs, MAC, timing visible	H	L	Cannot be hidden in normal networking	Accepted: reveals only which server is contacted
R6	ESP32 web page (8.5) read on the LAN	M	L	Read-only, shows no secrets	Accepted on own WPA2 network
R7	Print job (8.6) unencrypted on port 9100	L	L	Purpose is a readable paper copy; content is broadcast SSIDs; only printer + ESP32 on the WPA2 Wi-Fi Direct network; no credentials in the report (section 9)	Accepted
R8	Anyone with USB access changes or erases credentials	L	M	Physical access required	Accepted (issue I-4)
R9	MAC filtering relied on as protection	L	M	Not relied on; MACs are sent in clear and easy to fake	Accepted as extra barrier only

12. Issues and open items

ID	Issue	Proposed action
I-1	The internet check uses plain HTTP.	Switch to `https://www.gstatic.com/generate_204` with `WiFiClientSecure` and `setCACert()`; set time with NTP first.
I-2	The planned web page (8.5) must never expose secrets.	Smoke-test checks that the page and `/api/status` never contain the stored passwords.
I-3	The print job is unencrypted.	Accept (R7, section 9): the output is meant to be read on paper. Ensure the report never includes credentials.
I-4	The setup menu has no access control: anyone with a USB cable can overwrite or erase credentials (but not read passwords).	Accept for lab boards; a menu PIN or a physical button held at boot would raise the bar.
I-5	A credentials copy (`wifi_credentials copy.h`) once appeared untracked and was not covered by `.gitignore`.	Fixed: pattern widened to `include/wifi_credentials*.h`. Lesson: ignore patterns, not single names.
I-6	It has not been verified that no secret was ever committed.	Run `git log --all -- include/wifi_credentials.h` and search history (`git log -p -S "<password>"`); rotate the password if anything is found.
I-7	The XOR salt and algorithm are public.	Accepted by design (section 7.3); level 3 removes the dependency on secrecy.
I-8	The friggebod board will be unattended.	Put it on a separate guest/IoT network before deployment.

13. Conclusion

The most likely leak – credentials in the public repository – is closed: no secret exists in the source, the firmware file or any file on the PC. Credentials live only on the board, obfuscated so that they are not readable in a flash dump and do not work on another chip. The analysis also shows the limits: the obfuscation is not encryption, MAC-based XOR is unsuitable for transmission, and some data (DNS, metadata, local HTTP, the print job) is visible by design and accepted with a stated reason – the print job, for example, is meant to end up as readable text on paper. The next levels – AES-GCM with an eFuse key, and flash encryption with Secure Boot – are documented with their costs, and the open issues are listed with proposed actions.